University of Canberra
Faculty of Science and Technology

Software Technology 2
(7170)
Week 7 Tutorial & Computer Lab
Assignment 1: Individual Project
Part 4: Tree Data Structures (Binary Search Trees, AVL Trese, Red-Black Trees)

- This lab activity is assessed and contributes to the Assignment 1 (Individual Project) mark.
- Please go through Week 6 lecture recordings and Week_7_Tutorial_and Computer_Laboratory.doc available after Module 7 in Canvas (Assignment 1/Individual Project preparation activities)
- For assessment of this activity, you need to complete the following in week 6 tutorial/lab:

  1) attend your allocated computer lab class in Week 6,

  2) complete all tasks listed in this handout,

  3) demonstrate the working activities to your tutor in class, and

  4) Save your python project to a zip file and upload this zip file to **In-class Lab Work Demo Week 7** and upload it to your GitHub Repository. Make sure you include a commit message with date and time, when you upload this zip file in GitHub.
- Your mark is 0 if you do not demonstrate your project.
- You can leave the class after you complete the 4 requirements above.

Tasks

- Understanding **Linear Data Structures** (linked lists, stacks, queues and priority queues and their implementation in Python (**data_structures_practice.py**)
- **Benchmarking Performance** of Linear Data Structures (data_structures_practice.py)
- Prior Reading: **Week_6_Lecture** available in Module 6 in Canvas**.**

Download the **ST2 Week 7 Tutorial and Computer Lab** word document in Module 7 in Canvas. This document contains all the necessary activities needed for each task completion in this week.

Learning Objectives/Goals:

- Understand the concepts of **Tree** data structure and associated terminology.
- Implement basic tree structures in Python.
- Practice and Visualise tree traversals.
- Solve problems involving binary trees, binary search trees (BST), and other tree types.
- Implement an application involving trees.

Task 1: Design and Implementation of Binary Search Tree (BST) Data Structure

In this task, you will design and implement a simple Tree Data Structure – the Binary Search Tree. Follow each step below to create this simple Tree Data Structure and test it.

Step 1: Basic Tree Implementation in Python

- Tree terminology: Root, Node, Edge, Parent, Child, Leaf, Subtree, Depth, Height.
- Types of trees: Binary tree, Binary Search Tree, Balanced Trees, N-ary Tree.
- Create a Python file called **bst.py** and create the **Node** class as shown below:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

# Example: create root node
root = Node(10)
root.left = Node(5)
root.right = Node(15)
```

Step 2 : Add a method **Preorder Traversal (Root-Left-Right)** in the **Node** Class you have written in Step 1

```
def preorder(root):
    if root:
        print(root.value, end=' ')
        preorder(root.left)
        preorder(root.right)
```

Step 3: Add second method called **Inorder Traversal (Left-Root-Right)** to the Node Class.

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.value, end=' ')
        inorder(root.right)
```

Step 4: Add third method called **Postorder Traversal (Left-Right-Root)** to the Node Class.

```
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.value, end=' ')
```

Step 5: Add fourth method to the Node Class for counting total number of nodes in the tree called **count_nodes**.

```
def count_nodes(root):
    if root is None:
        return 0
    return 1 + count_nodes(root.left) + count_nodes(root.right)
```

Step 5: And the next method to the Node Class for finding the height of a tree called **height**.

```
def height(root):
    if root is None:
        return 0
    return 1 + max(height(root.left), height(root.right))
```

Step 7: Add the next method to the Node Class for searching for a value in the Binary Search Tree (BST). Name the method as **search_bst.**

```
def search_bst(root, val):
    if root is None or root.value == val:
        return root
```

```
    if val < root.value:
        return search_bst(root.left, val)
    else:
        return search_bst(root.right, val)
```

Step 8: Add the next method to the Node Class for inserting a value in the Binary Search Tree (BST). Name the method as **insert_bst.**

```
def insert_bst(root, val):
    if root is None:
        return Node(val)
    if val < root.value:
        root.left = insert_bst(root.left, val)
    else:
        root.right = insert_bst(root.right, val)
    return root
```

Step 9: Add the next method to the Node Class for finding the minimum value node in a BST. Name the method as **find_min.**

```
def find_min(root):
    current = root
    while current.left:
        current = current.left
    return current
```

Step 10: Add the next method to the Node Class for deleting node in a BST. Name the method as **delete_note.**

```
def delete_node(root, val):
    if root is None:
        return root
    if val < root.value:
        root.left = delete_node(root.left, val)
    elif val > root.value:
        root.right = delete_node(root.right, val)
    else:
        # Node with 1 or no child
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        # Node with two children
        temp = find_min(root.right)
        root.value = temp.value
        root.right = delete_node(root.right, temp.value)
    return root
```

Step 11: Add the next method to the Node Class for traversal/navigation in a BST. Use a level order traversal with breadth-first search. Name the method as **level_order.**

```
from collections import deque
def level_order(root):
    if not root:
        return
    queue = deque([root])
    while queue:
        node = queue.popleft()
        print(node.value, end=' ')
        if node.left:
            queue.append(node.left)
        if node.right:
```

```
            queue.append(node.right)
```

Step 12: The simple bst (**Binary Search Tree**) data structure is complete, Now you will implement this data structure for building an application- a **Contact Directory using BST**.
Create a contact directory where each contact has a name and phone number. The contacts should be stored in a BST keyed by contact name, insert, search and delete operations. Also add a test function to print all contacts in alphabetical order (using inorder traversal).
For this, you either need to create two classes Contact Class, ContactDirectory class or modify the bst_tree.py to customize the tree for Contact Directory application.

```
class Contact:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.left = None
        self.right = None

class ContactDirectory:
    def __init__(self):
        self.root = None

    def insert(self, root, name, phone):
        if root is None:
            return Contact(name, phone)
        if name < root.name:
            root.left = self.insert(root.left, name, phone)
        elif name > root.name:
            root.right = self.insert(root.right, name, phone)
        else:
            # Update phone if contact already exists
            root.phone = phone
        return root

    def search(self, root, name):
        if root is None or root.name == name:
            return root
        if name < root.name:
            return self.search(root.left, name)
        else:
            return self.search(root.right, name)

    def find_min(self, root):
        current = root
        while current.left:
            current = current.left
        return current

    def delete(self, root, name):
        if root is None:
            return root
        if name < root.name:
            root.left = self.delete(root.left, name)
        elif name > root.name:
            root.right = self.delete(root.right, name)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left
```

```
            temp = self.find_min(root.right)
            root.name = temp.name
            root.phone = temp.phone
            root.right = self.delete(root.right, temp.name)
        return root

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(f"Name: {root.name}, Phone: {root.phone}")
            self.inorder(root.right)

# Testing the binary search tree (
directory = ContactDirectory()
directory.root = directory.insert(directory.root, 'Alice', '123-456-
7890')
directory.root = directory.insert(directory.root, 'Bob', '234-567-
8901')
directory.root = directory.insert(directory.root, 'Eve', '345-678-
9012')

print("All contacts:")
directory.inorder(directory.root)

print("\nSearching for Bob:")
contact = directory.search(directory.root, 'Bob')
if contact:
    print(f"Found contact: {contact.name} - {contact.phone}")
else:
    print("Contact not found")

print("\nDeleting Alice:")
directory.root = directory.delete(directory.root, 'Alice')
directory.inorder(directory.root)
```

A successful test run of the code should display the following output. Show the program output of testing this ContactDirectory.

```
All contacts:
Name: Alice, Phone: 123-456-7890
Name: Bob, Phone: 234-567-8901
Name: Eve, Phone: 345-678-9012

Searching for Bob:
Found contact: Bob - 234-567-8901

Deleting Alice:
Name: Bob, Phone: 234-567-8901
Name: Eve, Phone: 345-678-9012
```

Task 2: BST Tree visualization and Benchmarking

Extend the BST Tree from Task 1 for the ContactDirectory with a GUI visualisation to:

- Insert contacts
- Search contacts
- Delete contacts

- Display all contacts in sorted order (Inorder traversal) in a GUI text widget
- Benchmark functionality to that measure insertion/search/deletion time using Python **time** module.

```
import tkinter as tk
from tkinter import messagebox
import time

class Contact:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.left = None
        self.right = None

class ContactDirectory:
    def __init__(self):
        self.root = None

    def insert(self, root, name, phone):
        if root is None:
            return Contact(name, phone)
        if name < root.name:
            root.left = self.insert(root.left, name, phone)
        elif name > root.name:
            root.right = self.insert(root.right, name, phone)
        else:
            # Update phone if already exists
            root.phone = phone
        return root

    def search(self, root, name):
        if root is None or root.name == name:
            return root
        if name < root.name:
            return self.search(root.left, name)
        else:
            return self.search(root.right, name)

    def find_min(self, root):
        current = root
        while current.left:
            current = current.left
        return current

    def delete(self, root, name):
        if root is None:
            return root
        if name < root.name:
            root.left = self.delete(root.left, name)
        elif name > root.name:
            root.right = self.delete(root.right, name)
        else:
            # Node with one or no child
            if root.left is None:
```

```
                return root.right
            elif root.right is None:
                return root.left
            # Node with two children
            temp = self.find_min(root.right)
            root.name = temp.name
            root.phone = temp.phone
            root.right = self.delete(root.right, temp.name)
        return root

    def inorder(self, root, result=None):
        if result is None:
            result = []
        if root:
            self.inorder(root.left, result)
            result.append((root.name, root.phone))
            self.inorder(root.right, result)
        return result


# ---- Tkinter GUI Visualiser ----

class BSTVisualizerApp:
    def __init__(self, root):
        self.directory = ContactDirectory()

        self.root = root
        self.root.title("Contact Directory BST")

        # Top frame for Inputs
        frame = tk.Frame(root)
        frame.pack(pady=10)

        tk.Label(frame, text="Name:").grid(row=0, column=0, padx=5)
        self.name_entry = tk.Entry(frame)
        self.name_entry.grid(row=0, column=1, padx=5)

        tk.Label(frame, text="Phone:").grid(row=1, column=0, padx=5)
        self.phone_entry = tk.Entry(frame)
        self.phone_entry.grid(row=1, column=1, padx=5)

        button_frame = tk.Frame(root)
        button_frame.pack()

        self.insert_btn = tk.Button(button_frame, text="Insert
Contact", command=self.insert_contact)
        self.insert_btn.grid(row=0, column=0, padx=5)

        self.search_btn = tk.Button(button_frame, text="Search
Contact", command=self.search_contact)
        self.search_btn.grid(row=0, column=1, padx=5)

        self.delete_btn = tk.Button(button_frame, text="Delete
Contact", command=self.delete_contact)
        self.delete_btn.grid(row=0, column=2, padx=5)

        self.show_all_btn = tk.Button(button_frame, text="Show All
Contacts", command=self.show_all)
```

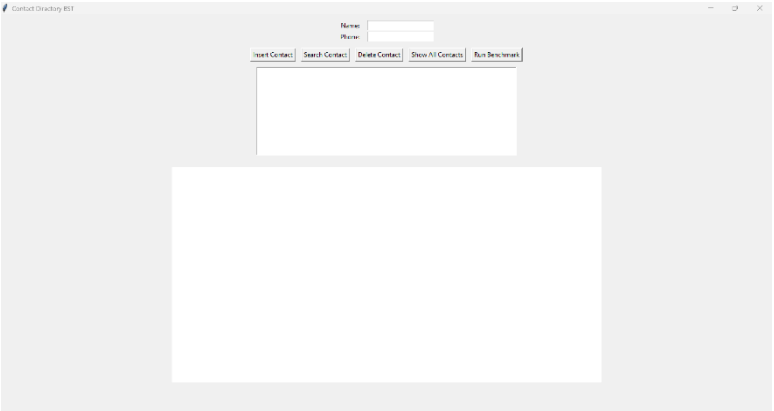
```
        self.show_all_btn.grid(row=0, column=3, padx=5)

        self.benchmark_btn = tk.Button(button_frame, text="Run
Benchmark", command=self.benchmark)
        self.benchmark_btn.grid(row=0, column=4, padx=5)

        # Area to display contacts
        self.output_text = tk.Text(root, height=10, width=60)
        self.output_text.pack(pady=10)

        # Canvas to visualize BST
        self.canvas = tk.Canvas(root, width=800, height=400,
bg='white')
        self.canvas.pack(pady=10)

    # GUI functions

    def insert_contact(self):
        name = self.name_entry.get().strip()
        phone = self.phone_entry.get().strip()
        if not name or not phone:
            messagebox.showwarning("Input Error", "Please enter both
name and phone number.")
            return
        self.directory.root =
self.directory.insert(self.directory.root, name, phone)
        messagebox.showinfo("Success", f"Inserted/Updated contact:
{name}")
        self.clear_entries()
        self.show_all()
        self.draw_tree()

    def search_contact(self):
        name = self.name_entry.get().strip()
        if not name:
            messagebox.showwarning("Input Error", "Please enter a
name to search.")
            return
        node = self.directory.search(self.directory.root, name)
        self.output_text.delete(1.0, tk.END)
        if node:
            self.output_text.insert(tk.END, f"Found Contact:\nName:
{node.name}\nPhone: {node.phone}\n")
        else:
            self.output_text.insert(tk.END, "Contact not found.")

    def delete_contact(self):
        name = self.name_entry.get().strip()
        if not name:
            messagebox.showwarning("Input Error", "Please enter a
name to delete.")
            return
        self.directory.root =
self.directory.delete(self.directory.root, name)
        messagebox.showinfo("Success", f"Deleted contact (if
existed): {name}")
        self.clear_entries()
        self.show_all()
```

```
        self.draw_tree()

    def show_all(self):
        self.output_text.delete(1.0, tk.END)
        contacts = self.directory.inorder(self.directory.root)
        if not contacts:
            self.output_text.insert(tk.END, "No contacts found.\n")
        else:
            for name, phone in contacts:
                self.output_text.insert(tk.END, f"Name: {name},
Phone: {phone}\n")

    def clear_entries(self):
        self.name_entry.delete(0, tk.END)
        self.phone_entry.delete(0, tk.END)

    # BST Visualization on Canvas

    def draw_tree(self):
        self.canvas.delete("all")
        if not self.directory.root:
            return
        start_x = self.canvas.winfo_width() // 2
        start_y = 20
        level_gap = 70
        node_radius = 20
        self._draw_node(self.directory.root, start_x, start_y,
self.canvas.winfo_width() // 4, level_gap, node_radius)

    def _draw_node(self, node, x, y, x_offset, level_gap, radius):
        if node is None:
            return

        # Draw left subtree
        if node.left:
            x_left = x - x_offset
            y_left = y + level_gap
            # line to left child
            self.canvas.create_line(x, y, x_left, y_left)
            self._draw_node(node.left, x_left, y_left, x_offset // 2,
level_gap, radius)

        # Draw right subtree
        if node.right:
            x_right = x + x_offset
            y_right = y + level_gap
            # line to right child
            self.canvas.create_line(x, y, x_right, y_right)
            self._draw_node(node.right, x_right, y_right, x_offset //
2, level_gap, radius)

        # Draw the current node (circle with name inside)
        self.canvas.create_oval(x - radius, y - radius, x + radius, y
+ radius, fill="lightblue")
        display_name = node.name if len(node.name) <= 10 else
node.name[:10] + "..."
        self.canvas.create_text(x, y, text=display_name)
```

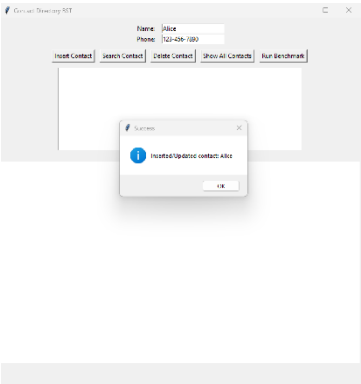
```
    # Benchmarking

    def benchmark(self):
        # For benchmarking, generate a large number of random
contacts
        import random
        import string

        # Clear any existing contacts
        self.directory.root = None

        N = 1000  # Number of contacts
        names = [''.join(random.choices(string.ascii_lowercase, k=7))
for _ in range(N)]
        phones = [''.join(random.choices(string.digits, k=10)) for _
in range(N)]

        # Benchmark Insertions
        start = time.time()
        for i in range(N):
            self.directory.root =
self.directory.insert(self.directory.root, names[i], phones[i])
        insert_time = time.time() - start

        # Benchmark Searches (search half of inserted names + some
random)
        search_names = random.sample(names, N//2) + ['notfoundname']*
(N//2)
        start = time.time()
        found_count = 0
        for name in search_names:
            node = self.directory.search(self.directory.root, name)
            if node:
                found_count += 1
        search_time = time.time() - start

        # Benchmark Deletions (delete 100 random contacts)
        delete_names = random.sample(names, 100)
        start = time.time()
        for name in delete_names:
            self.directory.root =
self.directory.delete(self.directory.root, name)
        delete_time = time.time() - start

        result_text = (
            f"Benchmark Results (N={N}):\n"
            f"Insertion time: {insert_time:.4f} seconds\n"
            f"Search time (for {len(search_names)} queries, found
{found_count}): {search_time:.4f} seconds\n"
            f"Deletion time (for 100 deletions): {delete_time:.4f}
seconds\n"
        )

        self.output_text.delete(1.0, tk.END)
        self.output_text.insert(tk.END, result_text)

        # Redraw tree after deletions
        self.draw_tree()
```

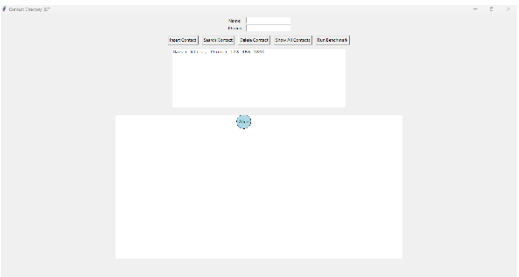
```
if __name__ == "__main__":
    root = tk.Tk()
    app = BSTVisualizerApp(root)
    root.mainloop()
```

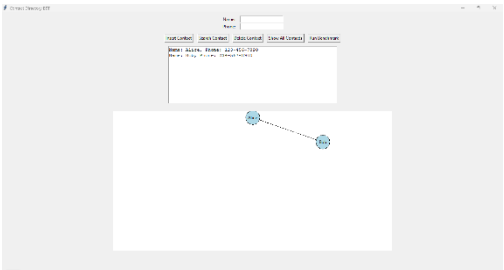
Test each function and display the output displayed/printed for this tree visualizer program (insert contact, search contact, delete contact, show all contacts, and run benchmark functions).



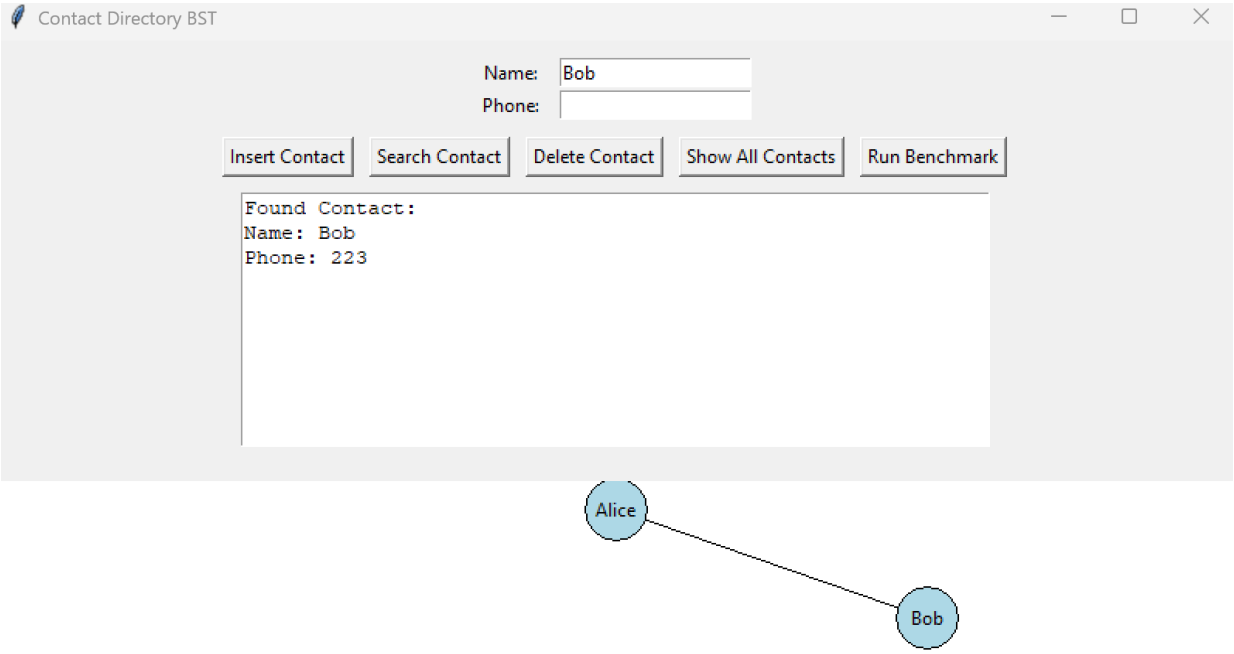
Insert



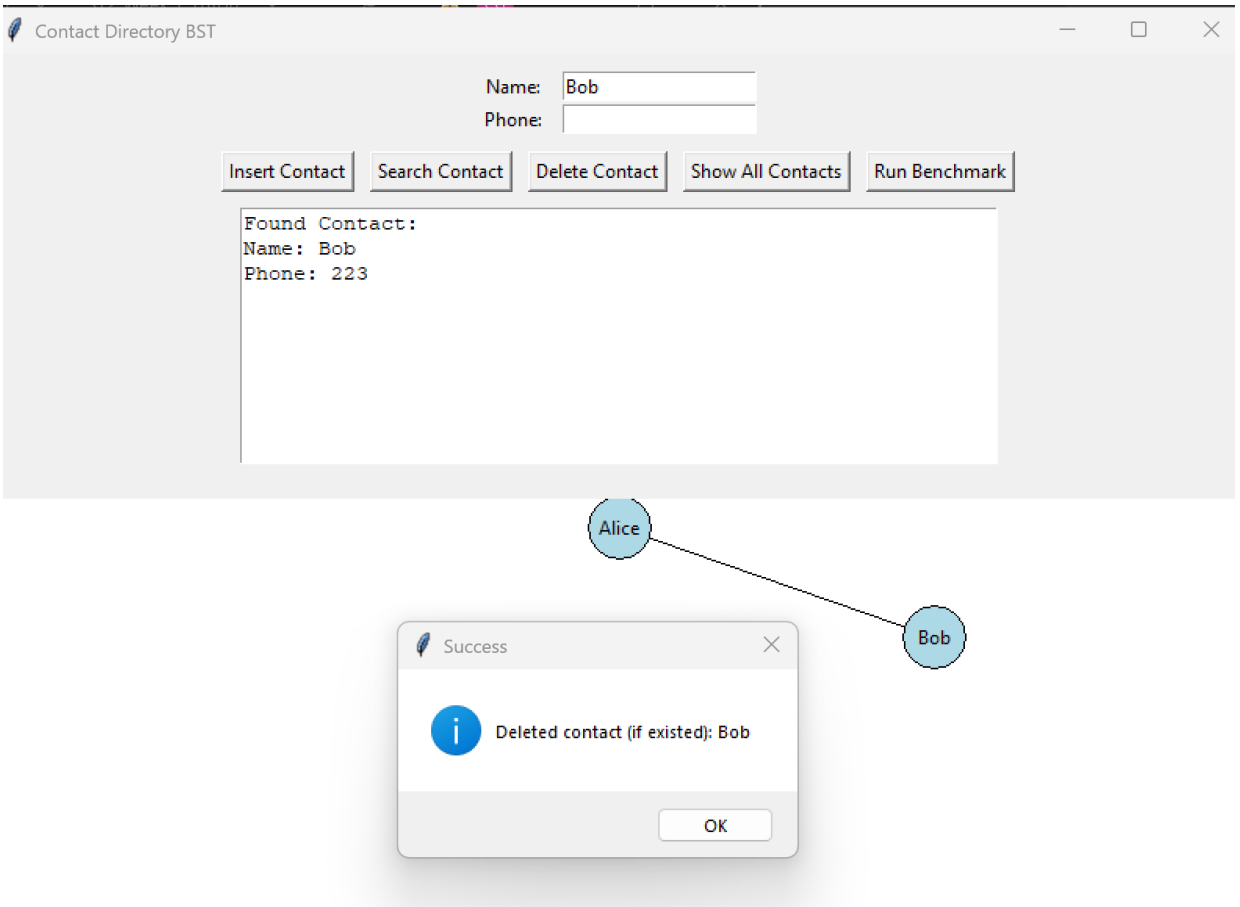


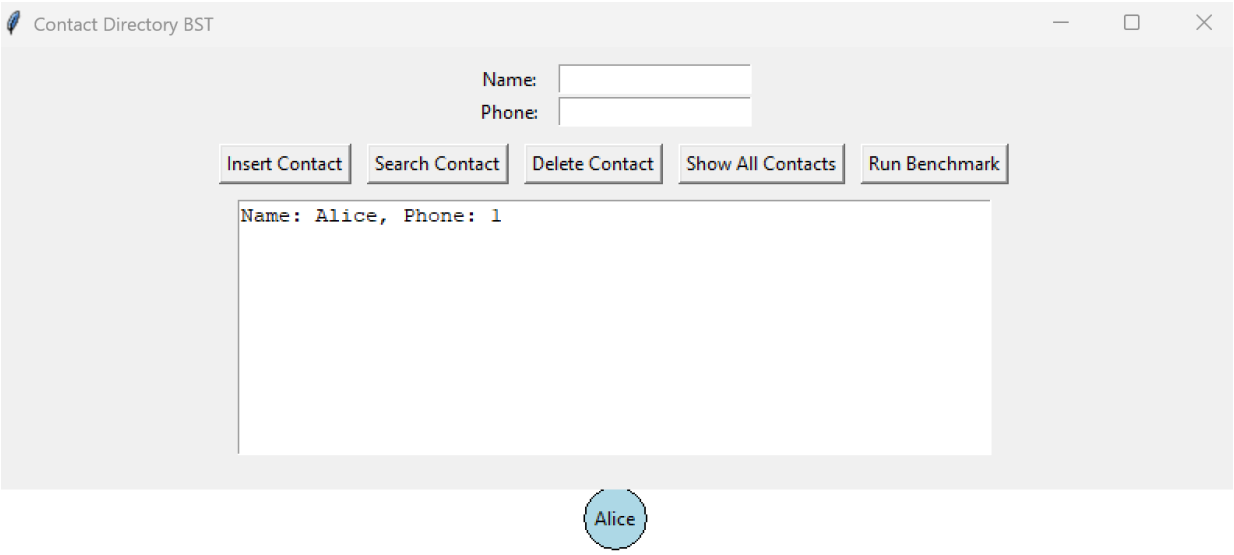


Search

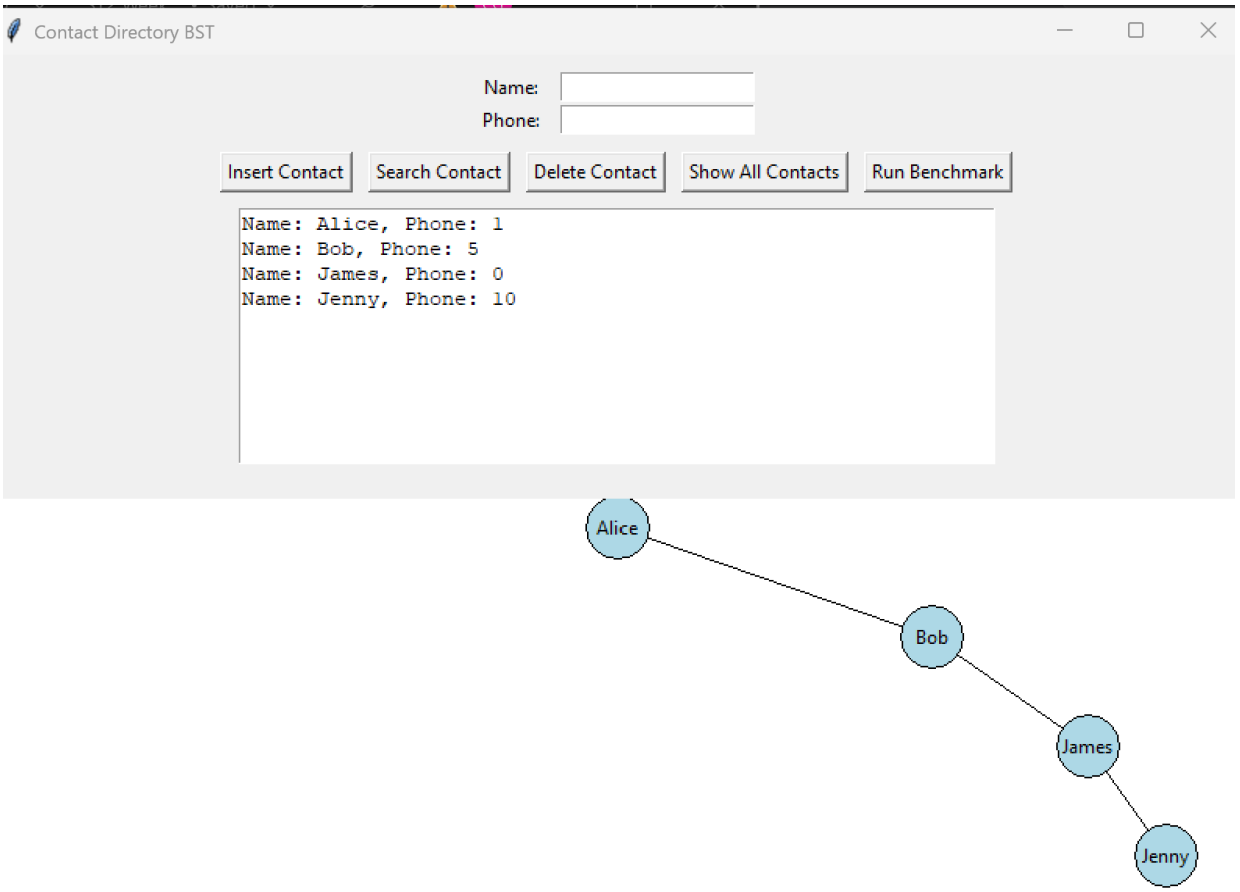

Contact Directory BST
Name:
Bob
Phone:
Insert Contact
Search Contact
Delete Contact
Show All Contacts
Run Benchmark
Found Contact:
Name: Bob
Phone: 223
Alice
Bob

Delete

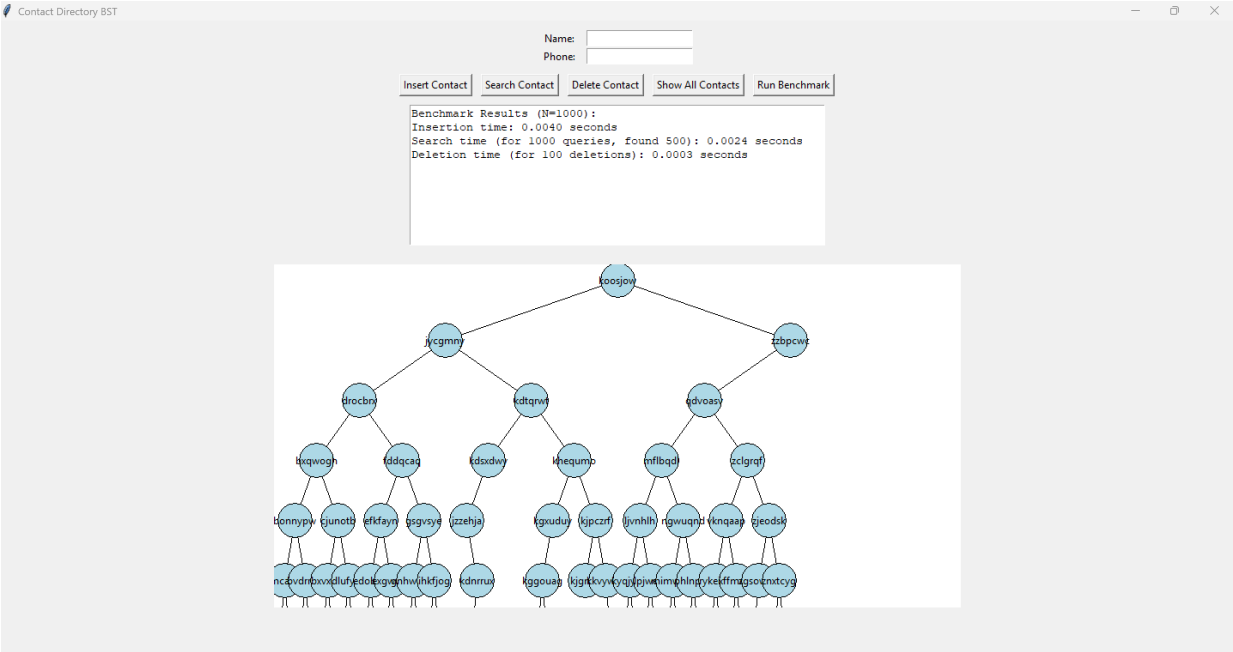

Contact Directory BST
Name:
Bob
Phone:
Insert Contact
Search Contact
Delete Contact
Show All Contacts
Run Benchmark
Found Contact:
Name: Bob
Phone: 223
Alice
Bob
Success
Deleted contact (if existed): Bob
OK



Show Contacts



Run Benchmarks



Task 3: Design and Implementation of Balanced Binary Search Trees (AVL and Red-Black Trees)

In this task you will design a **Balanced Binary Search Tree**, such as **AVL tree** and **Red-Black Trees**.

Why Balanced BSTs?

- Problem with plain BST: Without balancing, BSTs can degrade to a linear chain (like a linked list), causing operations to degrade from O(log n) to O(n).
- Balanced BST keeps tree height as small as possible, guaranteeing O(log n) time for insert/search/delete.

---

1. AVL Trees

- Named after inventors Adelson-Velsky and Landis.
- Self-balancing BST where the height difference (balance factor) between left and right subtrees is at most 1 for every node.
- If insertion/deletion violates this, rotations are performed to restore balance.

Core concepts:

- Balance Factor = height(left subtree) - height(right subtree)
- Allowed values: -1, 0, 1
- Rotations:
  - Left rotation
  - Right rotation
  - Left-Right rotation
  - Right-Left rotation

Follow the steps shown below to **create a AVL tree** and test it.

Step 1: Design AVLNode class

Create the AVLNode class

```
class AVLNode:
    def __init__(self, key):
        self.key = key
```

```
        self.left = None
        self.right = None
        self.height = 1  # New node is initially added at leaf
```

Step 2: AVL Class Operations

Add following methods to AVL class for balancing the tree:

```
def height(node):
  if not node:
    return 0
  return node.height

def get_balance(node):
  if not node:
    return 0
  return height(node.left) - height(node.right)

def right_rotate(z):
  y = z.left
  T3 = y.right

  # Perform rotation
  y.right = z
  z.left = T3

  # Update heights
  z.height = 1 + max(height(z.left), height(z.right))
  y.height = 1 + max(height(y.left), height(y.right))

  # Return new root
  return y

def left_rotate(z):
  y = z.right
  T2 = y.left

  # Perform rotation
  y.left = z
  z.right = T2

  # Update heights
  z.height = 1 + max(height(z.left), height(z.right))
  y.height = 1 + max(height(y.left), height(y.right))

  # Return new root
  return y

def insert_avl(node, key):
  # 1. Perform standard BST insert
  if not node:
    return AVLNode(key)
  elif key < node.key:
    node.left = insert_avl(node.left, key)
  else:
    node.right = insert_avl(node.right, key)

  # 2. Update height
```

```
    node.height = 1 + max(height(node.left), height(node.right))

    # 3. Get balance factor
    balance = get_balance(node)

    # 4. Check for imbalance and do rotations

    # Left Left
    if balance > 1 and key < node.left.key:
        return right_rotate(node)

    # Right Right
    if balance < -1 and key > node.right.key:
        return left_rotate(node)

    # Left Right
    if balance > 1 and key > node.left.key:
        node.left = left_rotate(node.left)
        return right_rotate(node)

    # Right Left
    if balance < -1 and key < node.right.key:
        node.right = right_rotate(node.right)
        return left_rotate(node)

    return node
```

Step 3: AVL Visualiser for ContactDirectory

Extend the AVL tree for **ContactDictionary App** with GUI Interface supporting name-entry contacts (using AVL Tree), with:

- Insert contact (with balancing)
- Delete contact (with balancing)
- Search contact
- Show all contacts (inorder)
- Visualize tree structure and rotations on canvas

```
import tkinter as tk
from tkinter import messagebox
import time

# === AVL Tree Node ===

class AVLNode:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.left = None
        self.right = None
        self.height = 1

# === AVL Tree Class ===

class AVLTree:
    def __init__(self):
```

```
    self.root = None

def height(self, node):
    if not node:
        return 0
    return node.height

def get_balance(self, node):
    if not node:
        return 0
    return self.height(node.left) - self.height(node.right)

def right_rotate(self, z):
    y = z.left
    T3 = y.right

    # Rotation
    y.right = z
    z.left = T3

    # Update heights
    z.height = 1 + max(self.height(z.left), self.height(z.right))
    y.height = 1 + max(self.height(y.left), self.height(y.right))

    return y

def left_rotate(self, z):
    y = z.right
    T2 = y.left

    # Rotation
    y.left = z
    z.right = T2

    # Update heights
    z.height = 1 + max(self.height(z.left), self.height(z.right))
    y.height = 1 + max(self.height(y.left), self.height(y.right))

    return y

def insert(self, node, name, phone):
    # Normal BST insert
    if not node:
        return AVLNode(name, phone)
    if name < node.name:
        node.left = self.insert(node.left, name, phone)
    elif name > node.name:
        node.right = self.insert(node.right, name, phone)
    else:
        # Update phone if name exists
        node.phone = phone
        return node

    # Update height
    node.height = 1 + max(self.height(node.left), self.height(node.right))
```

```
    # Check balance
    balance = self.get_balance(node)

    # Left Left
    if balance > 1 and name < node.left.name:
        return self.right_rotate(node)

    # Right Right
    if balance < -1 and name > node.right.name:
        return self.left_rotate(node)

    # Left Right
    if balance > 1 and name > node.left.name:
        node.left = self.left_rotate(node.left)
        return self.right_rotate(node)

    # Right Left
    if balance < -1 and name < node.right.name:
        node.right = self.right_rotate(node.right)
        return self.left_rotate(node)

    return node

def min_value_node(self, node):
    current = node
    while current.left is not None:
        current = current.left
    return current

def delete(self, root, name):
    # Standard BST delete
    if not root:
        return root
    if name < root.name:
        root.left = self.delete(root.left, name)
    elif name > root.name:
        root.right = self.delete(root.right, name)
    else:
        # Node with one or zero children
        if root.left is None:
            temp = root.right
            root = None
            return temp
        elif root.right is None:
            temp = root.left
            root = None
            return temp
        # Node with two children
        temp = self.min_value_node(root.right)
        root.name = temp.name
        root.phone = temp.phone
        root.right = self.delete(root.right, temp.name)

    if root is None:
        return root
```

```
        # Update height
        root.height = 1 + max(self.height(root.left), self.height(root.right))

        # Check balance
        balance = self.get_balance(root)

        # Left Left
        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        # Left Right
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # Right Right
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        # Right Left
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def search(self, node, name):
        if node is None or node.name == name:
            return node
        if name < node.name:
            return self.search(node.left, name)
        else:
            return self.search(node.right, name)

    def inorder(self, root, res=None):
        if res is None:
            res = []
        if root:
            self.inorder(root.left, res)
            res.append((root.name, root.phone))
            self.inorder(root.right, res)
        return res

# === Tkinter GUI App ===

class AVLApp:
    def __init__(self, master):
        self.tree = AVLTree()

        self.master = master
        master.title("AVL Contact Directory")

        # Input frame
        self.frame = tk.Frame(master)
        self.frame.pack(pady=10)
```

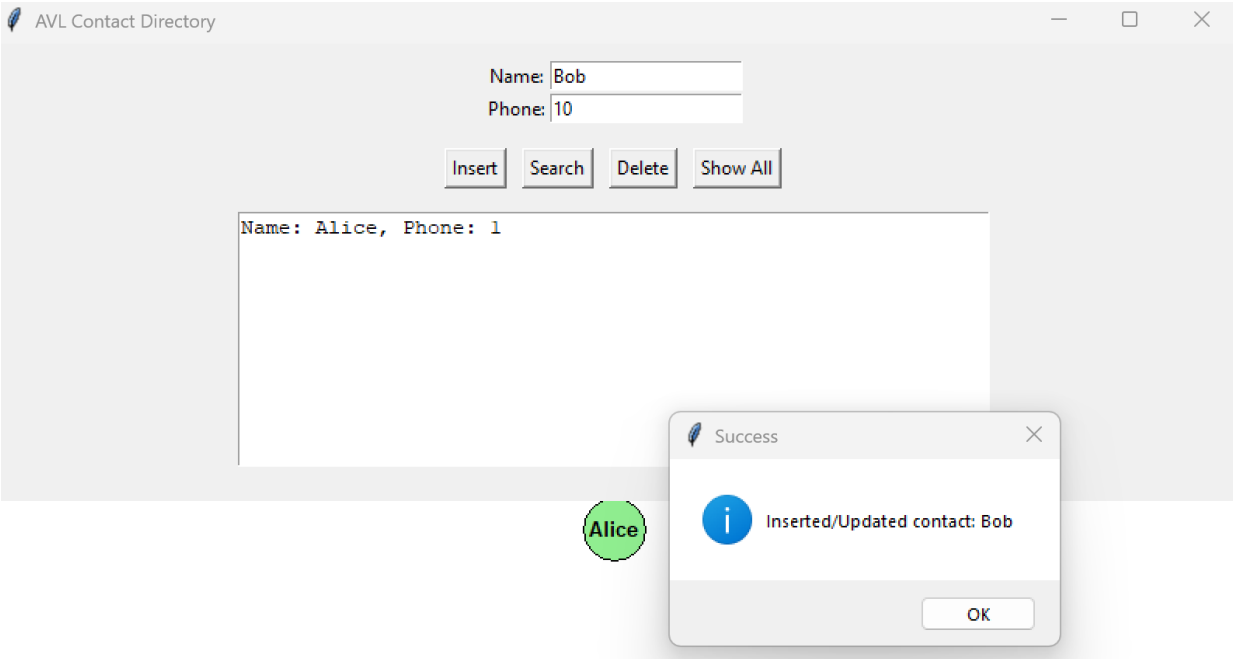
```
        tk.Label(self.frame, text="Name:").grid(row=0, column=0)
        self.name_entry = tk.Entry(self.frame)
        self.name_entry.grid(row=0, column=1)

        tk.Label(self.frame, text="Phone:").grid(row=1, column=0)
        self.phone_entry = tk.Entry(self.frame)
        self.phone_entry.grid(row=1, column=1)

        # Buttons
        self.btn_frame = tk.Frame(master)
        self.btn_frame.pack(pady=5)

        tk.Button(self.btn_frame, text="Insert", command=self.insert_contact).grid(row=0,
column=0, padx=5)
        tk.Button(self.btn_frame, text="Search", command=self.search_contact).grid(row=0,
column=1, padx=5)
        tk.Button(self.btn_frame, text="Delete", command=self.delete_contact).grid(row=0,
column=2, padx=5)
        tk.Button(self.btn_frame, text="Show All", command=self.show_all).grid(row=0,
column=3, padx=5)

        # Output text box
        self.output_text = tk.Text(master, height=10, width=60)
        self.output_text.pack(pady=10)

        # Canvas for tree visualization
        self.canvas = tk.Canvas(master, width=800, height=400, bg="white")
        self.canvas.pack(pady=10)

    def insert_contact(self):
        name = self.name_entry.get().strip()
        phone = self.phone_entry.get().strip()
        if not name or not phone:
            messagebox.showwarning("Input Error", "Please enter both Name and Phone.")
            return
        self.tree.root = self.tree.insert(self.tree.root, name, phone)
        messagebox.showinfo("Success", f"Inserted/Updated contact: {name}")
        self.clear_entries()
        self.show_all()
        self.draw_tree()

    def search_contact(self):
        name = self.name_entry.get().strip()
        if not name:
            messagebox.showwarning("Input Error", "Please enter Name to search.")
            return
        node = self.tree.search(self.tree.root, name)
        self.output_text.delete(1.0, tk.END)
        if node:
            self.output_text.insert(tk.END, f"Found Contact:\nName: {node.name}\nPhone:
{node.phone}\n")
        else:
            self.output_text.insert(tk.END, "Contact not found.\n")

    def delete_contact(self):
        name = self.name_entry.get().strip()
```

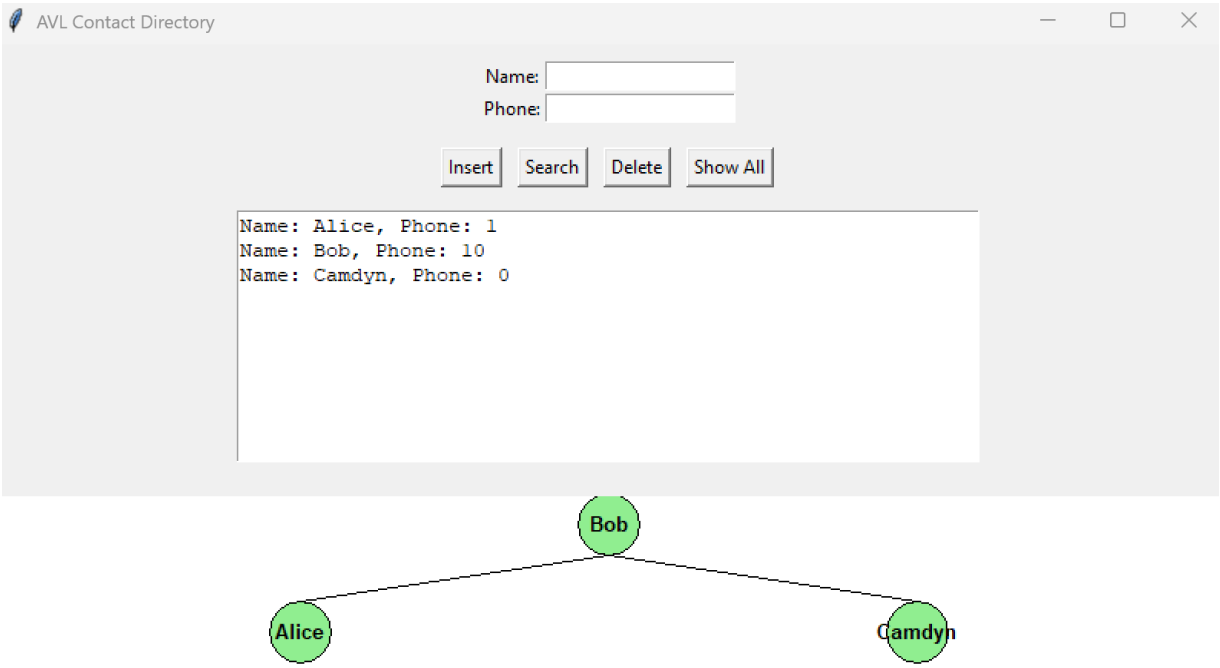
```
    if not name:
        messagebox.showwarning("Input Error", "Please enter Name to delete.")
        return
    self.tree.root = self.tree.delete(self.tree.root, name)
    messagebox.showinfo("Success", f"Deleted contact (if existed): {name}")
    self.clear_entries()
    self.show_all()
    self.draw_tree()

def show_all(self):
    self.output_text.delete(1.0, tk.END)
    contacts = self.tree.inorder(self.tree.root)
    if not contacts:
        self.output_text.insert(tk.END, "No contacts found.\n")
    else:
        for n, p in contacts:
            self.output_text.insert(tk.END, f"Name: {n}, Phone: {p}\n")

def clear_entries(self):
    self.name_entry.delete(0, tk.END)
    self.phone_entry.delete(0, tk.END)

def draw_tree(self):
    self.canvas.delete("all")
    if not self.tree.root:
        return

    width = self.canvas.winfo_width()
    height = self.canvas.winfo_height()
    self._draw_node(self.tree.root, width // 2, 20, width // 4)

def _draw_node(self, node, x, y, x_offset):
    if not node:
        return

    radius = 20
    # Draw left subtree + edge
    if node.left:
        x_left = x - x_offset
        y_left = y + 70
        self.canvas.create_line(x, y+radius, x_left, y_left - radius)
        self._draw_node(node.left, x_left, y_left, x_offset // 2)

    # Draw right subtree + edge
    if node.right:
        x_right = x + x_offset
        y_right = y + 70
        self.canvas.create_line(x, y+radius, x_right, y_right - radius)
        self._draw_node(node.right, x_right, y_right, x_offset // 2)

    # Draw node circle
    self.canvas.create_oval(x - radius, y - radius, x + radius, y + radius, fill="lightgreen")

    # Draw text - shortened name if long
    display_name = node.name if len(node.name) <= 10 else node.name[:10] + "..."
    self.canvas.create_text(x, y, text=display_name, font=("Arial", 10, "bold"))
```

```
# Run the app
if __name__ == "__main__":
    root = tk.Tk()
    app = AVLApp(root)
    root.mainloop()
```

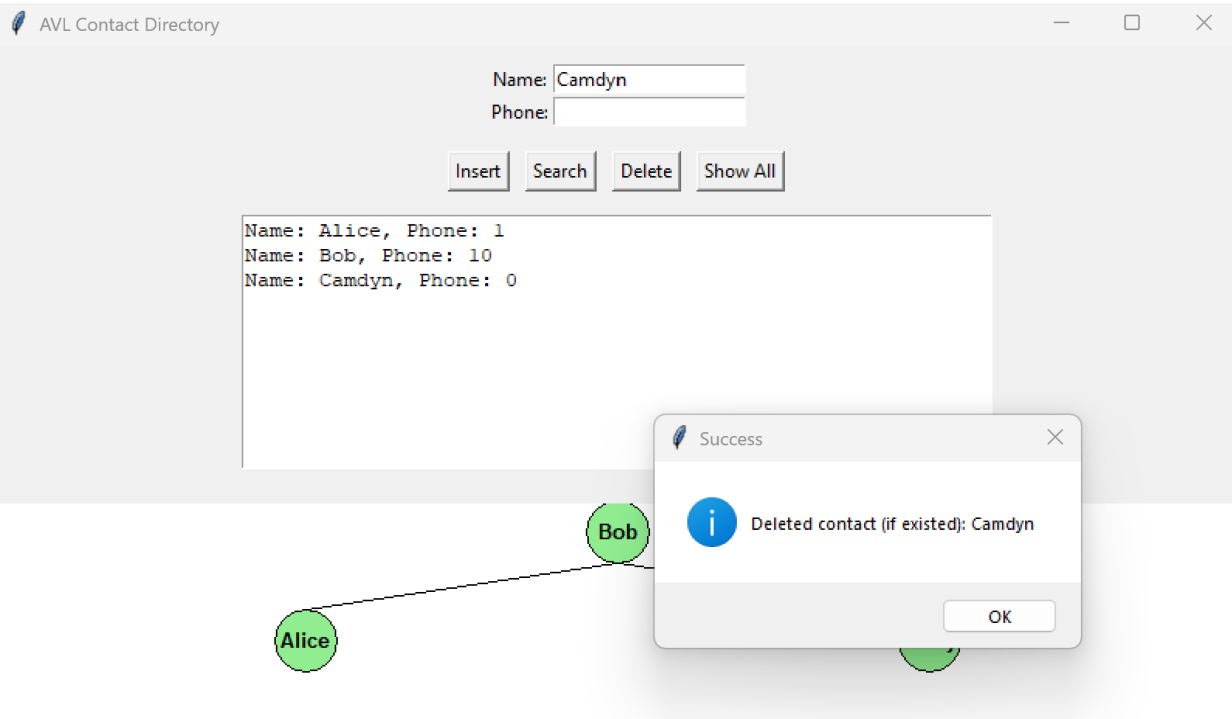
Test AVLTree GUI for ContactDictionary with similar examples as for BST and show the results(screenshots) for each insert, search and delete operation, and how rotation operation balances the tree.

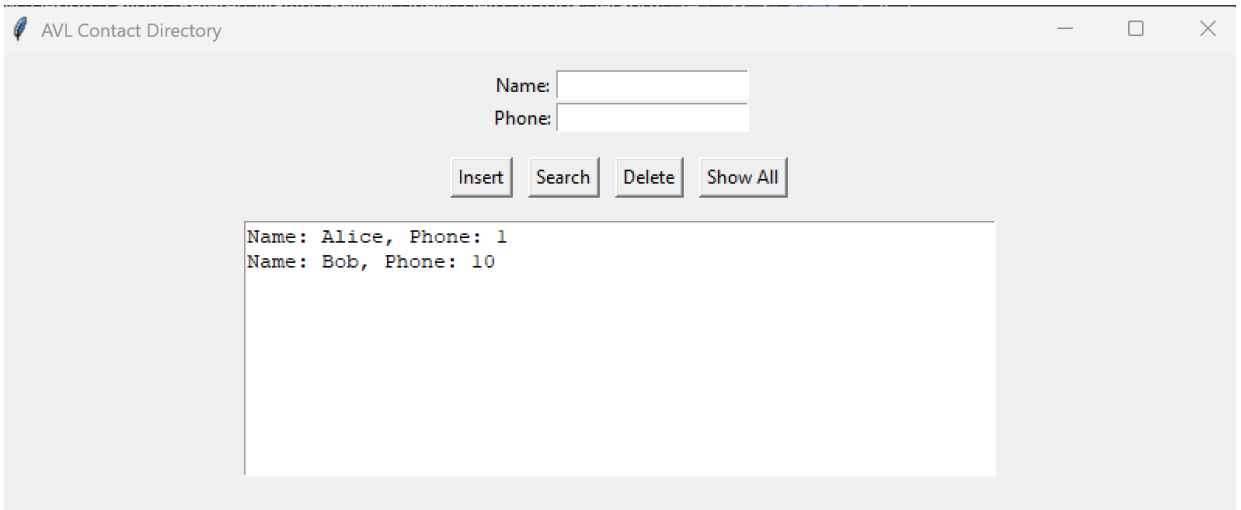
Insert and Show All

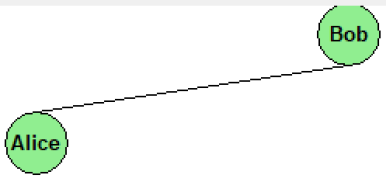



AVL Contact Directory
Name:
Phone:
Insert
Search
Delete
Show All
Name: Alice, Phone: 1
Name: Bob, Phone: 10
Name: Camdyn, Phone: 0
Bob
Alice
Camdyn

Delete


AVL Contact Directory
Name: Camdyn
Phone:
Insert
Search
Delete
Show All
Name: Alice, Phone: 1
Name: Bob, Phone: 10
Name: Camdyn, Phone: 0
Success
Deleted contact (if existed): Camdyn
OK
Bob
Alice





<u>Task 4: Benchmarking AVL Tree with Binary Search Trees(BSTs)</u>

In this task you will compare the time complexities of standard **Binary Search Trees (BSTs)** and **AVL Tree**s, which will help you understand why balanced trees are important for maintaining efficient operations.

Time Complexity Comparison: BST vs AVL Tree

Operation	Average Time Complexity	Worst-case Time Complexity	Notes
BST (Unbalanced)	O(log n)	O(n)	In average cases with random inserts, BST operations are logarithmic. In worst cases (e.g., sorted inputs), BST becomes skewed (like linked list), degrading to linear time.
AVL Tree	O(log n)	O(log n)	AVL trees maintain strict balance, guaranteeing logarithmic height, so all operations are always O(log n). The tree performs rotations to keep balanced after insert/delete.

Details:

1. Binary Search Tree (BST)

- Average case:
  When keys are inserted in a random order, the tree tends to be balanced approximately, so height ≈ $\log_2 n$
  Operations (search, insert, delete) traverse height, leading to O(log n) time.
- Worst case:
  When keys are inserted in sorted order (ascending or descending), all nodes are added to one side:
  The tree height becomes n (linear chain).
  Operations degrade to O(n) — unacceptable for large datasets.

---

2. AVL Tree

- Height guarantee:
  The AVL tree maintains a height bounded by:
  (h \leq 1.44 \log_2(n + 2) - 0.328) (approximately) — always logarithmic.
- Operations:
  Search: follows a path from root to leaf → O(log n)
  Insert: standard BST insert + up to O(log n) rotations → still O(log n)
  Delete: standard BST delete + balancing → O(log n)
- Rotation overhead:
  Rotations (single/double) are constant-time operations, performed at most O(log n) times during insert/delete.

Why AVL Trees Are Better for Performance-Critical Applications

- Consistent performance: Operations never degrade below O(log n), regardless of input order.
- Better than worst-case BST: For datasets with many insertions/deletions/searches, AVL guarantees speed.
- Tradeoff: Slightly more complex insertion/deletion logic; a bit more overhead due to balancing.

Summary

```
Balanced AVL Tree              Skewed BST (Worst Case)
  Height ≈ log n               Height = n
        o                          o
       / \                        /
      o   o                      o
     /     \                    /
    o       o                  o
   /                          /
  o                          o
```

- Operations are proportional to tree height, so minimizing height is crucial.

Practical Implementation Guidelines:

- For small datasets, normal BSTs may be enough.
- For large or dynamic datasets, balanced trees like AVL or Red-Black Tree are preferred.
- Balanced trees underpin many real-world data structures: databases, file systems, language libraries.
- Benchmarking Python script that compares insertion, search, and deletion runtimes between:
  - Standard BST (unbalanced)
  - AVL Tree (balanced)
  - It prints the results in the console and also displays a simple Tkinter GUI showing the benchmark summary.
  - The GUI part can be used for quick visualization.

```
import time
import random
import string
import tkinter as tk
from tkinter import ttk

# ===== BST Node and BST Class =====
```

```
class Node:
    def __init__(self, name, phone=None):
        self.name = name
        self.phone = phone
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, root, name, phone):
        if root is None:
            return Node(name, phone)
        if name < root.name:
            root.left = self.insert(root.left, name, phone)
        elif name > root.name:
            root.right = self.insert(root.right, name, phone)
        else:
            root.phone = phone
        return root

    def search(self, root, name):
        if root is None or root.name == name:
            return root
        if name < root.name:
            return self.search(root.left, name)
        else:
            return self.search(root.right, name)

    def find_min(self, root):
        current = root
        while current.left:
            current = current.left
        return current

    def delete(self, root, name):
        if root is None:
            return root
        if name < root.name:
            root.left = self.delete(root.left, name)
        elif name > root.name:
            root.right = self.delete(root.right, name)
        else:
            # Node with one or no children
            if root.left is None:
                temp = root.right
                root = None
                return temp
            elif root.right is None:
                temp = root.left
                root = None
                return temp
            # Node with two children
            temp = self.find_min(root.right)
```

```
        root.name = temp.name
        root.phone = temp.phone
        root.right = self.delete(root.right, temp.name)
    return root

# ===== AVL Node and AVL Tree Class =====

class AVLNode:
  def __init__(self, name, phone):
    self.name = name
    self.phone = phone
    self.left = None
    self.right = None
    self.height = 1

class AVLTree:
  def __init__(self):
    self.root = None

  def height(self, node):
    return node.height if node else 0

  def get_balance(self, node):
    if not node:
      return 0
    return self.height(node.left) - self.height(node.right)

  def right_rotate(self, z):
    y = z.left
    T3 = y.right

    y.right = z
    z.left = T3

    z.height = 1 + max(self.height(z.left), self.height(z.right))
    y.height = 1 + max(self.height(y.left), self.height(y.right))
    return y

  def left_rotate(self, z):
    y = z.right
    T2 = y.left

    y.left = z
    z.right = T2

    z.height = 1 + max(self.height(z.left), self.height(z.right))
    y.height = 1 + max(self.height(y.left), self.height(y.right))
    return y

  def insert(self, node, name, phone):
    if not node:
      return AVLNode(name, phone)
    if name < node.name:
      node.left = self.insert(node.left, name, phone)
    elif name > node.name:
      node.right = self.insert(node.right, name, phone)
```

```
        else:
            node.phone = phone
            return node

        node.height = 1 + max(self.height(node.left), self.height(node.right))
        balance = self.get_balance(node)

        # Rotations
        if balance > 1 and name < node.left.name:
            return self.right_rotate(node)
        if balance < -1 and name > node.right.name:
            return self.left_rotate(node)
        if balance > 1 and name > node.left.name:
            node.left = self.left_rotate(node.left)
            return self.right_rotate(node)
        if balance < -1 and name < node.right.name:
            node.right = self.right_rotate(node.right)
            return self.left_rotate(node)

        return node

    def min_value_node(self, node):
        current = node
        while current.left:
            current = current.left
        return current

    def delete(self, root, name):
        if not root:
            return root
        if name < root.name:
            root.left = self.delete(root.left, name)
        elif name > root.name:
            root.right = self.delete(root.right, name)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left
            temp = self.min_value_node(root.right)
            root.name = temp.name
            root.phone = temp.phone
            root.right = self.delete(root.right, temp.name)

        root.height = 1 + max(self.height(root.left), self.height(root.right))
        balance = self.get_balance(root)

        # Balancing
        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)
        if balance < -1 and self.get_balance(root.right) > 0:
```

```
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    def search(self, node, name):
        if node is None or node.name == name:
            return node
        if name < node.name:
            return self.search(node.left, name)
        else:
            return self.search(node.right, name)

# ===== Benchmarking Function =====

def benchmark(n=1000, search_fraction=0.5, delete_count=100, verbose=True):
    # Generate test data
    names = [''.join(random.choices(string.ascii_lowercase, k=10)) for _ in range(n)]
    phones = [''.join(random.choices(string.digits, k=10)) for _ in range(n)]

    # --- BST benchmark ---
    bst = BST()
    bst.root = None

    # Insert
    start = time.time()
    for i in range(n):
        bst.root = bst.insert(bst.root, names[i], phones[i])
    bst_insert_time = time.time() - start

    # Search
    search_names = random.sample(names, int(n*search_fraction)) + ['notfoundname'] *
int(n*(1 - search_fraction))
    start = time.time()
    found_bst = sum(1 for name in search_names if bst.search(bst.root, name))
    bst_search_time = time.time() - start

    # Delete
    delete_names = random.sample(names, delete_count)
    start = time.time()
    for name in delete_names:
        bst.root = bst.delete(bst.root, name)
    bst_delete_time = time.time() - start

    # --- AVL benchmark ---
    avl = AVLTree()
    avl.root = None

    # Insert
    start = time.time()
    for i in range(n):
        avl.root = avl.insert(avl.root, names[i], phones[i])
    avl_insert_time = time.time() - start

    # Search
    start = time.time()
```

```
    found_avl = sum(1 for name in search_names if avl.search(avl.root, name))
    avl_search_time = time.time() - start

    # Delete
    start = time.time()
    for name in delete_names:
        avl.root = avl.delete(avl.root, name)
    avl_delete_time = time.time() - start

    # Prepare results
    results = {
        'Insertions': (bst_insert_time, avl_insert_time),
        'Searches': (bst_search_time, avl_search_time),
        'Deletions': (bst_delete_time, avl_delete_time),
        'Found In Search': (found_bst, found_avl)
    }

    if verbose:
        print(f"Benchmarking with {n} entries:\n")
        print("Operation     BST Time (sec)    AVL Time (sec)")
        print("---------------------------------------------")
        print(f"Insertions:   {bst_insert_time:.6f}          {avl_insert_time:.6f}")
        print(f"Searches:     {bst_search_time:.6f}          {avl_search_time:.6f}")
        print(f"Deletions:    {bst_delete_time:.6f}          {avl_delete_time:.6f}")
        print("---------------------------------------------")
        print(f"Contacts found in search:")
        print(f"BST: {found_bst} / {len(search_names)}")
        print(f"AVL: {found_avl} / {len(search_names)}")
    return results

# ===== Tkinter GUI to show benchmark results =====

class BenchmarkApp:
    def __init__(self, master):
        self.master = master
        master.title("BST vs AVL Benchmark")

        self.label = tk.Label(master, text="Run benchmark with 1000 entries?")
        self.label.pack(pady=5)

        self.run_btn = tk.Button(master, text="Run Benchmark",
command=self.run_benchmark)
        self.run_btn.pack(pady=5)

        self.result_text = tk.Text(master, height=15, width=50)
        self.result_text.pack(pady=10)

        self.tree = ttk.Treeview(master, columns=("BST", "AVL"), show='headings')
        self.tree.heading("BST", text="BST Time (s)")
        self.tree.heading("AVL", text="AVL Time (s)")
        self.tree.pack(pady=10)

    def run_benchmark(self):
        self.result_text.delete(1.0, tk.END)
        results = benchmark(verbose=False)  # silent print
```

```
    # Display text summary
    summary = (
      f"Insertion Time: BST: {results['Insertions'][0]:.6f}s, AVL:
{results['Insertions'][1]:.6f}s\n"
      f"Search Time:   BST: {results['Searches'][0]:.6f}s, AVL: {results['Searches'][1]:.6f}s\n"
      f"Deletion Time:  BST: {results['Deletions'][0]:.6f}s, AVL:
{results['Deletions'][1]:.6f}s\n"
      f"Found in Search: BST: {results['Found In Search'][0]}, AVL: {results['Found In
Search'][1]} out of 1000\n"
    )
    self.result_text.insert(tk.END, summary)

    # Clear previous treeview
    for row in self.tree.get_children():
      self.tree.delete(row)

    # Insert new data rows
    for op in ['Insertions', 'Searches', 'Deletions']:
      bst_time, avl_time = results[op]
      self.tree.insert('', tk.END, values=(f"{bst_time:.6f}", f"{avl_time:.6f}"), text=op)

# ===== Run the app =====

if __name__ == "__main__":
  root = tk.Tk()
  app = BenchmarkApp(root)
  root.mainloop()
```

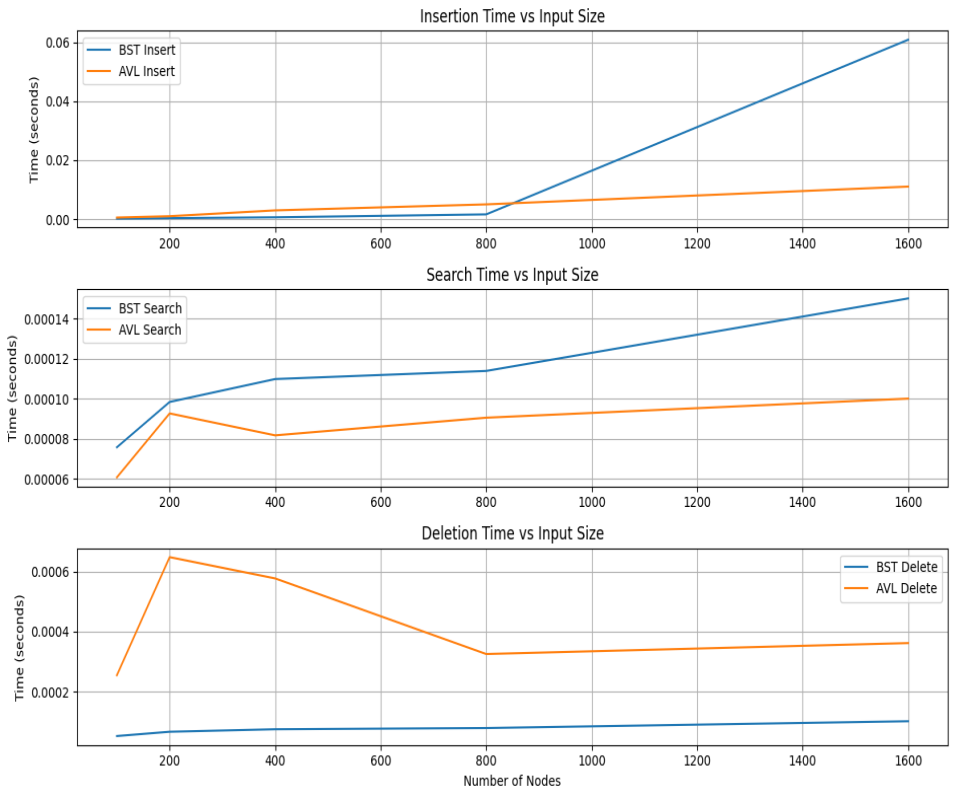
- Run the benchmarking app comparing AVL tree with BST, and show the output obtained for different data sizes. Currently it shows the performance for a data size of 1000.
- Provide enhanced visualisation for different data sizes by adding the following code fragment to the existing code for this task as shown below:

```
import time
import random
import string
import matplotlib.pyplot as plt

# Re-use BST and AVL classes from previous code
# Use BST and AVLTree classes already defined with
# insert/search/delete methods.

def run_benchmark_for_sizes(sizes, num_search=100,
num_delete=50):
    bst_times = {'insert': [], 'search': [], 'delete': []}
    avl_times = {'insert': [], 'search': [], 'delete': []}

    for n in sizes:
        names = [''.join(random.choices(string.ascii_lowercase,
k=8)) for _ in range(n)]
        phones = [''.join(random.choices(string.digits, k=10))
for _ in range(n)]

        search_names = random.sample(names, min(num_search, n))
        delete_names = random.sample(names, min(num_delete, n))

        # --- BST ---
```

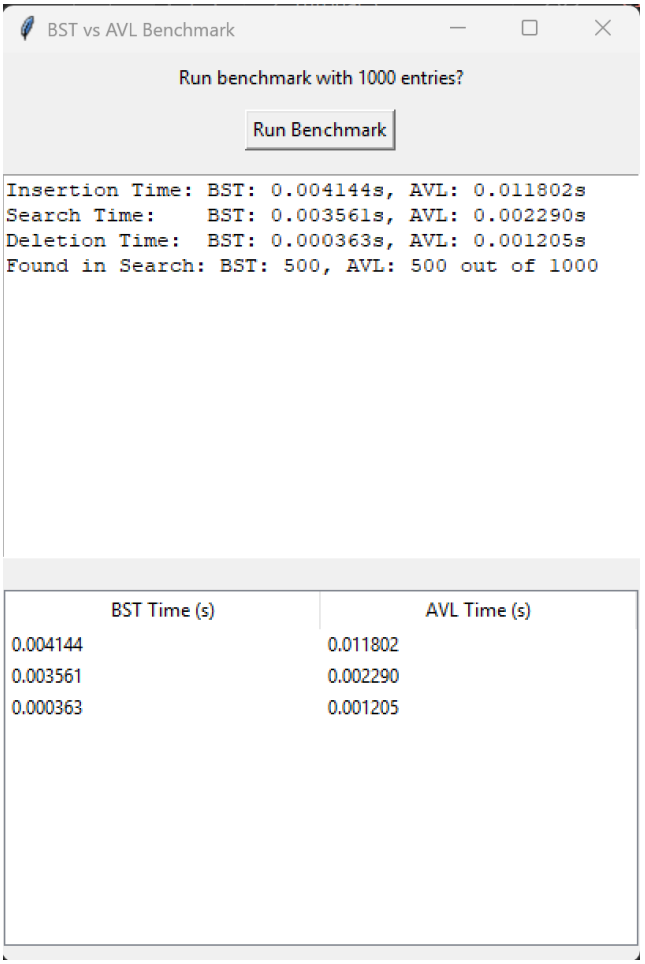
```
    bst = BST()
    start = time.time()
    for i in range(n):
        bst.root = bst.insert(bst.root, names[i], phones[i])
    bst_times['insert'].append(time.time() - start)

    start = time.time()
    for name in search_names:
        bst.search(bst.root, name)
    bst_times['search'].append(time.time() - start)

    start = time.time()
    for name in delete_names:
        bst.root = bst.delete(bst.root, name)
    bst_times['delete'].append(time.time() - start)

    # --- AVL ---
    avl = AVLTree()
    start = time.time()
    for i in range(n):
        avl.root = avl.insert(avl.root, names[i], phones[i])
    avl_times['insert'].append(time.time() - start)

    start = time.time()
    for name in search_names:
        avl.search(avl.root, name)
    avl_times['search'].append(time.time() - start)

    start = time.time()
    for name in delete_names:
        avl.root = avl.delete(avl.root, name)
    avl_times['delete'].append(time.time() - start)

# Plot results
plt.figure(figsize=(12, 8))

# Insertions plot
plt.subplot(3, 1, 1)
plt.plot(sizes, bst_times['insert'], label='BST Insert')
plt.plot(sizes, avl_times['insert'], label='AVL Insert')
plt.ylabel('Time (seconds)')
plt.title('Insertion Time vs Input Size')
plt.legend()

# Searches plot
plt.subplot(3, 1, 2)
plt.plot(sizes, bst_times['search'], label='BST Search')
plt.plot(sizes, avl_times['search'], label='AVL Search')
plt.ylabel('Time (seconds)')
plt.title('Search Time vs Input Size')
plt.legend()

# Deletions plot
plt.subplot(3, 1, 3)
plt.plot(sizes, bst_times['delete'], label='BST Delete')
plt.plot(sizes, avl_times['delete'], label='AVL Delete')
plt.xlabel('Number of Nodes')
plt.ylabel('Time (seconds)')
```

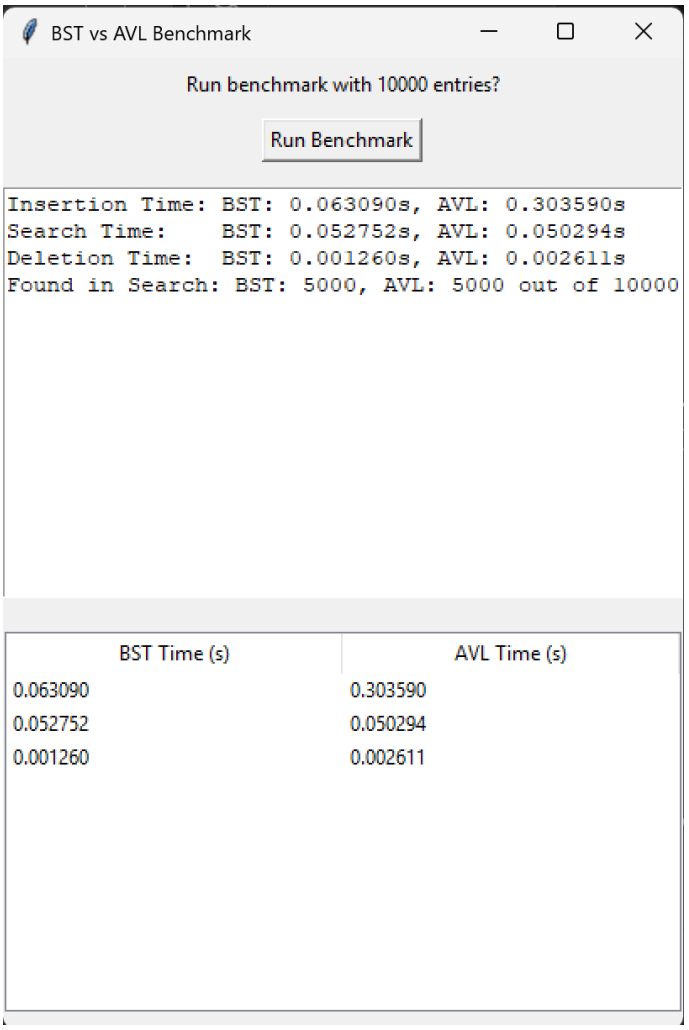
```
    plt.title('Deletion Time vs Input Size')
    plt.legend()

    plt.tight_layout()
    plt.show()

# Example usage:
if __name__ == "__main__":
    sizes_to_test = [100, 200, 400, 800, 1600]
    run_benchmark_for_sizes(sizes_to_test)
```

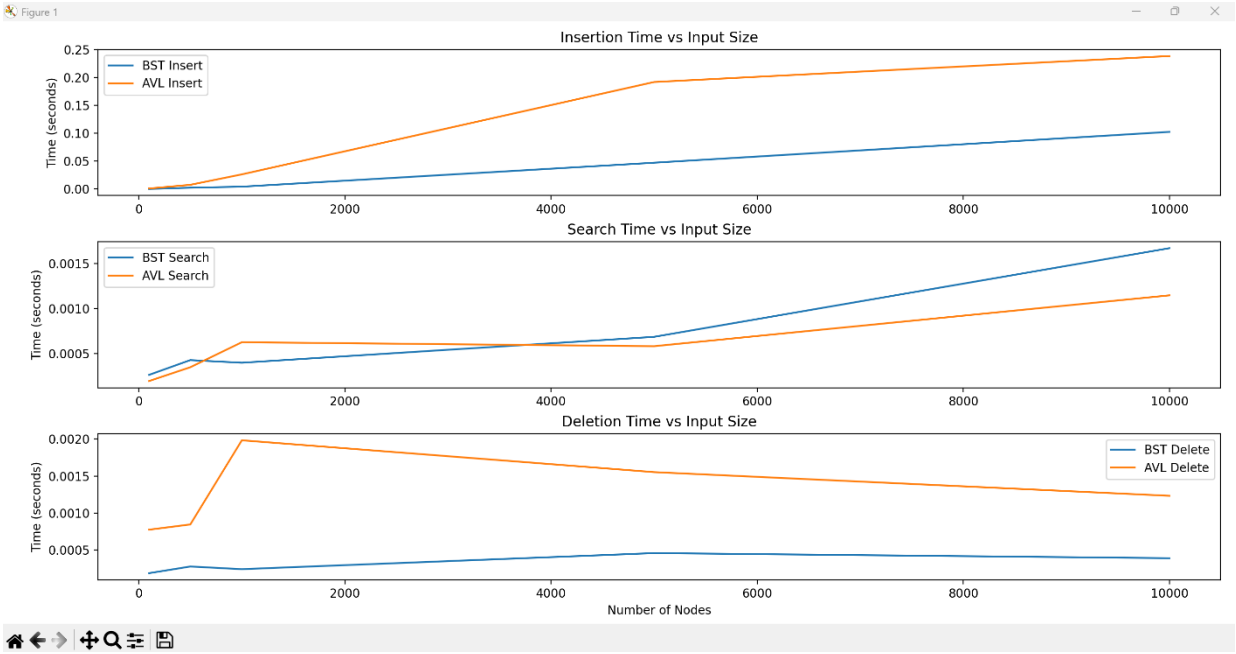
This should display the following results:



- Repeat the observations above with **increased data sizes (~ upto 10000)** and show the results. Discuss if your observations in relation to different operations (insert/search/delete) validate some of the following general facts about AVL vs. BSTs.
    - Insertion: AVL may be slightly slower than BST due to rotations, but avoids worst-case degradation.
    - Search: AVL generally performs faster or equal since the tree stays balanced.
    - Deletion: AVL deletion involves balancing, so minor overhead, but maintains performance.
    - Found count: Should nearly always match for both.
    - How BST runtime for insert/search/delete can degrade on some data orders, whereas AVL always stays consistent.
    - For completely random data, BST and AVL might not differ drastically, but for real-world or sorted data sets AVL shines.
    - AVL's stricter balancing ensures log(n) height always.
    - AVL rotations cost little compared to benefits for large data sets.


BST vs AVL Benchmark
Run benchmark with 1000 entries?
Run Benchmark
Insertion Time: BST: 0.004144s, AVL: 0.011802s
Search Time: BST: 0.003561s, AVL: 0.002290s
Deletion Time: BST: 0.000363s, AVL: 0.001205s
Found in Search: BST: 500, AVL: 500 out of 1000
BST Time (s)
AVL Time (s)
0.004144
0.011802
0.003561
0.002290
0.000363
0.001205


BST vs AVL Benchmark
Run benchmark with 10000 entries?
Run Benchmark
Insertion Time: BST: 0.063090s, AVL: 0.303590s
Search Time: BST: 0.052752s, AVL: 0.050294s
Deletion Time: BST: 0.001260s, AVL: 0.002611s
Found in Search: BST: 5000, AVL: 5000 out of 10000
BST Time (s)
AVL Time (s)
0.063090
0.303590
0.052752
0.050294
0.001260
0.002611



As the number of nodes increased up to 10,000, the results generally supported the expected theoretical differences between the Binary Search Tree (BST) and AVL Tree. Insertion was consistently slightly slower in the AVL tree due to the extra balancing operations and rotations required to maintain an optimal structure. However, this overhead helps prevent worst-case performance degradation that can occur in a BST when data is inserted in an already sorted order. Because AVL trees maintain a balanced height, their search performance remained consistently equal to or slightly faster than the BST, particularly as the dataset grew larger. This demonstrates how AVL trees maintain O(log n) time complexity, while BST performance can degrade toward O(n) in unfavorable data orders.

Deletion operations in the AVL tree also showed slightly higher execution time due to rebalancing, but this ensures the structure remains efficient for future operations. The number of successful search results (found count) was the same for both trees, confirming both structures correctly stored and retrieving the same data. Overall, while BST and AVL performance did not differ drastically with random data, the AVL tree demonstrated more consistent performance and scalability. This supports the conclusion that AVL trees are generally more reliable for larger or real-world datasets where data ordering may negatively affect BST efficiency.

Task 5 : Challenge Task : Design, Implement, Visualise and Benchmark the Red-Black-Tree.

Extend the Code from Task 3 and 4, and include Red-Black Tree demonstrating the design, implementation, visualization and benchmarking of Red-Black-Trees, and comparing with BST and AVL trees.
Here are some basic concepts of Red-Black-Trees

1.Red-Black Trees

- Another kind of self-balancing BST.
- Nodes have an extra bit for color - red or black.
- Maintains balance by constraining colour patterns along paths.
- Properties:
    1. Each node is red or black.
    2. Root is black.

3. Red nodes cannot have red children (no two reds in a row).
4. Every path from a node to a leaf has same number of black nodes.

- Insertion and deletion included recoloring and rotations.
- Commonly used in language libraries (e.g., Java's TreeMap, C++ STL map).

2. When to Use Balanced Trees?

- When frequent inserts/deletes and searches, balanced trees guarantee good performance.
- AVL is stricter balanced than Red-Black (slightly faster lookups).
- Red-Black often faster for insert/delete due to fewer rotations.

3. Comparison

Tree Type	**Search Time**	**Insert/Delete Time**	**Balance Method**	**Notes**
BST (unbalanced)	O(n) worst	O(n) worst	None	Can become skewed
AVL Tree	O(log n)	O(log n)	Rotations on imbalance	Strict balance, more rotations
Red-Black Tree	O(log n)	O(log n)	Color rules + rotation	Looser balance, fewer rotations

---

4. Resources / Further Reading

- AVL Tree Wiki
- Red-Black Tree Wiki